Langfuse LLM Observability

Integration Report — Smart City Dynamic Dispatch Grid

What changed, why it is useful, how to use it, and its limits.

1. What is Langfuse and why add it here?

Langfuse is an **LLM observability platform**. It records every call the application makes to a language model as a *trace* you can inspect in a dashboard: the exact prompt sent, the completion returned, the model used, token counts, cost, latency, and any errors.

This project has exactly **one** place that calls an LLM — the LLM-backed triage agent that turns raw 911 transcripts into structured `TriageReport`s (`dispatch_grid/llm_triage.py`). Everything else (dedup, quarantine, priority queue, dispatch, routing) is deterministic Python and needs no LLM tracing.

Before this change, that LLM call was a black box: if an extraction was wrong or silently fell back to the rule-based extractor, there was no way to see *what the model actually received and returned*. Langfuse makes that call observable.

Why it is useful

Benefit	What it gives you
Debugging	See the real prompt + model output for any batch — instantly explains a bad severity, a hallucinated location, or a fallback.
Cost & token tracking	Every call logs input/output token counts, so you can see and budget spend.
Latency monitoring	Per-call latency is recorded — spot slow batches.
Quality / evaluation	Inspect where the LLM disagrees with ground truth, batch by batch, in the evaluation harness.
Error visibility	Failed/blocked responses are tagged `ERROR` with a reason, instead of vanishing into a silent fallback.

2. What changed in the codebase

Four files were touched. **No existing behavior changed** — the integration is fully optional and additive.

2.1 `dispatch_grid/llm_triage.py` — the core change

(a) Optional Langfuse tracing.

- A lazy, cached helper `_langfuse()` initializes Langfuse **only if** the `langfuse` package is installed *and* both `LANGFUSE_PUBLIC_KEY` and `LANGFUSE_SECRET_KEY` are set. Otherwise it returns `None` and every trace call is a no-op — the extractor behaves exactly as before.
- `_call_llm()` wraps each batch request in a Langfuse **generation** via `_start_trace()` / `_end_trace()`, capturing the system prompt + input transcripts, model name, `max_tokens`, **token usage**, latency, the returned rows, and an `ERROR` level + message on failure.
- Token usage is now parsed from the API response (it wasn't captured before).

- Every tracing path is wrapped in `try/except` so a Langfuse failure **can never break triage** — matching the file's "degrade, don't halt" philosophy.
- `atexit.register(client.flush)` ensures short-lived CLI runs upload spans before exit.

(b) A pluggable LLM provider (Anthropic *or* Gemini). Because Langfuse is provider-agnostic, the call site was generalized to run against either backend, selected by `LLM_PROVIDER`:

- `_api_key()`, `_max_tokens()`, `_build_request()`, `_parse_response()` branch on the provider.
- **Anthropic** (default): unchanged request shape (`/v1/messages`, `x-api-key`).
- **Gemini**: Google's `generativelanguage` endpoint with `x-goog-api-key`, `responseMimeType: application/json` for clean structured output, and a larger output-token budget (Gemini "flash" is a *thinking* model that spends output tokens reasoning before it answers).

The public contract is untouched: `extract(call) -> TriageReport` and `extract_batch(...)` work identically; coordinator, dedup, and dispatch require no changes.

2.2 `requirements.txt`

Added `langfuse>=3` with a note that it is optional and safe to leave installed (tracing no-ops without keys).

2.3 `.env.sh` (new, git-ignored)

A convenience file holding the `export` lines for the Langfuse keys and LLM provider config. Git-ignored so real keys are never committed. You `source .env.sh` before running.

2.4 `.gitignore`

Added `.env.sh` so the secrets file can never be committed.

3. How to use it

3.1 Configure (one time) — fill real keys in `.env.sh`

```
# --- Langfuse (free) ---
export LANGFUSE_PUBLIC_KEY="pk-lf-..."       # Langfuse -> Settings -> API Keys
export LANGFUSE_SECRET_KEY="sk-lf-..."       # pk- = public, sk- = secret
export LANGFUSE_HOST="https://cloud.langfuse.com"   # US: https://us.cloud.langfuse.com

# --- LLM provider (Gemini free tier) ---
export LLM_PROVIDER="gemini"
export GEMINI_API_KEY="..."                  # Google AI Studio
export GEMINI_MODEL="gemini-3.6-flash"
```

To use Anthropic instead: `export LLM_PROVIDER="anthropic"` and set `ANTHROPIC_API_KEY` (optionally `TRIAGE_MODEL`).

3.2 Run something that uses the LLM

```
source .env.sh
python -m dispatch_grid.evaluation --llm --n 50
```

Confirm it worked from the tail of the output:

```
"agent_stats": { "llm_ok": 50, "fallbacks": 0, "api_calls": 5 }
```

`llm_ok > 0` and `api_calls > 0` means the LLM actually ran (and traces were sent). `llm_ok: 0` / `api_calls: 0` means it never ran — check your key.

3.3 View the traces

Open **cloud.langfuse.com → your project → Tracing → Traces**. You'll see `triage-extract` entries. Click one to see the prompt, the model's JSON output, model name, token usage, cost, latency, and metadata (`agent_id`, `batch_size`, `provider`).

4. Verification (what we observed)

A live run confirmed the whole path end-to-end on the Gemini free tier:

- **Provider/model:** `gemini-3.6-flash`
- **Extraction sanity** (3-call smoke test):
  - `"HUGE FIRE at Central Mall, people trapped"` → Fire, severity 5, location *Central Mall*, confidence 0.95 ✓
  - `"my friend said maybe theres smoke ... not sure"` (hedged secondhand) → confidence 0.20, below the 0.42 quarantine gate as intended ✓
  - `"car crash on 5th and Main, two people hurt"` → Accident, severity 2 ✓
- **Eval run** (`--n 8`): `llm_ok: 8, fallbacks: 0, api_calls: 1`, **no `401`/error lines** — every extraction came from the LLM and all spans were accepted by Langfuse.
- **Langfuse auth:** `Langfuse().auth_check()` → `True`.

5. Design principles

1. **Optional & non-breaking.** No keys or package → tracing is a silent no-op; extraction is byte-for-byte unchanged.
2. **Failure-isolated.** Any tracing error is caught and swallowed — observability never takes down dispatch.
3. **Provider-agnostic.** Langfuse traces whatever model runs; the same code traces Anthropic or Gemini identically.
4. **Single call site.** All instrumentation lives in `llm_triage.py`; the rest of the swarm is untouched.

6. Important limitation — the live web app is NOT traced (yet)

The deployed website and default simulation use the **rule-based** triage agent (`TriageAgent`), **not** the LLM one (`server/engine.py` builds `[TriageAgent(...) for i in range(4)]`). Real-city mode bypasses text triage entirely.

> **Consequence:** opening the deployed site produces **zero** Langfuse traces, because nothing there calls an LLM. Traces appear only when the LLM path runs — i.e. the `evaluation --llm` command. To trace the live app, its engine must be switched to `LLMTriageAgent` with the Gemini/Langfuse env vars set on the server (Render). That makes every run call the LLM (real latency/quota), so it was intentionally left as an opt-in and not enabled here.

7. File-change summary

File	Change	Commit?
`dispatch_grid/llm_triage.py`	Optional Langfuse tracing + pluggable Anthropic/Gemini provider	Yes
`requirements.txt`	Added `langfuse>=3` (optional dep)	Yes
`.gitignore`	Added `.env.sh`	Yes
`.env.sh`	Local secrets/config (git-ignored)	**No — never commit**